

区间问题

一：只查询不修改

1: n 个数, m 次查询, 求前缀和

2: n 个数, m 次查询, 求区间 $[a, b]$ 之和

3: n 个数, m 次查询, 求前 i 个数的极值

4: n 个数, m 次查询, 求区间 $[a, b]$ 的极值

5: n 个数, m 次查询, 求固定长度的所有区间的极值

区间问题

一：边查询边修改

- 1：点修改，求点的值
- 2：点修改，求区间 $[a, b]$ 之和
- 3：点修改，求区间的极值
- 4：区间修改，求点的值
- 5：区间修改，求区间的极值或者和

区间问题

当涉及到点修改区间求和的时候，我们可以先考虑直接修改和前缀修改的方式。

(1) 直接修改

修改时间复杂度： $O(1)$

查询时间复杂度： $O(n)$

(2) 前缀和修改

修改时间复杂度： $O(n)$

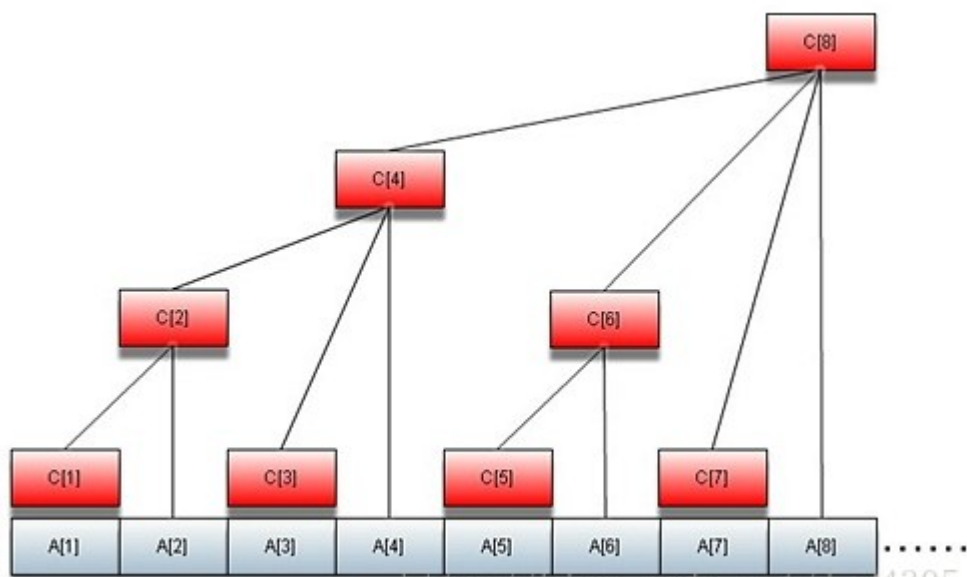
查询时间复杂度： $O(1)$

所以我们只需要把这两种方式综合一下，让修改和查询时间复杂度都变为 $\log n$ 就可以了。这就是树状数组的由来。

树状数组表现形式

树状数组入门博客：

<https://www.cnblogs.com/hsd-/p/6139376.html>



$C[1]=A[1]$
 $C[2]=A[1]+A[2]$
 $C[3]=A[3];$
 $C[4]=A[1]+\cdots A[4];$
 $C[5]=A[5];$
 $C[6]=A[5]+A[6];$
 $C[7]=A[7];$
 $C[8]=A[1]+\cdots A[8];$

树状数组原理

写成二进制形式

$$C[0001]=A[1]$$

$$C[0010]=A[1]+A[2]$$

$$C[0011]=A[3];$$

$$C[0100]=A[1]+\cdots A[4];$$

$$C[0101]=A[5];$$

$$C[0110]=A[5]+A[6];$$

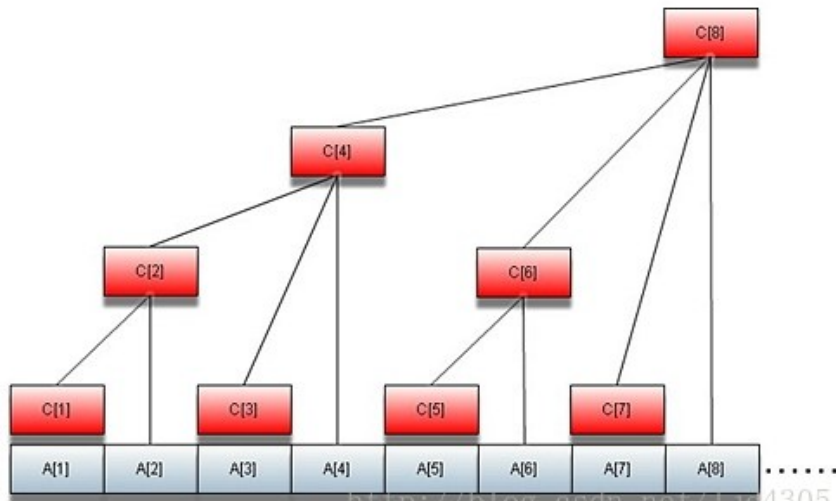
$$C[0111]=A[7];$$

$$C[1000]=A[8]+\cdots a[1];$$

规律：把十进制数转换成对应的二进制数，如果末尾是1，则 $C[i]$ 只包含它本身，如果末尾有一个0，则包含 $C[i], C[i-1]$ 。如果有两个0，则包含 $C[i]-C[i-3]$ ，如果有3个0，则包含 $C[i]-C[i-7]$ ，以此类推。

树状数组原理

所以，在计算的时候，如果我们要求前 i 个数之和。就可以根据 i 的大小以及 $C[k]$ 的值来确定。



$sum[0001] = C[1];$
 $sum[0010] = C[2];$
 $Sum[0011] = C[3] + C[2];$
 $Sum[0100] = C[4];$
 $Sum[0101] = C[5] + C[4];$
 $Sum[0110] = C[6] + C[4];$
 $Sum[0111] = C[7] + C[6] + C[4];$
 $Sum[1000] = C[8]$

规律：

$$sum[0111] = sum[0110] + C[0111]$$

$$Sum[0110] = sum[0100] + C[0110]$$

$$Sum[0100] = C[0100]$$

和最后一个1的位置有关，最后一个1在第 i 位，就减去 2^{i-1} 。

树状数组原理

所以如果要求区间 $[n, m]$ 之和，我们可以先转换成求 $\text{sum}[m]$ 与 $\text{sum}[n-1]$ 之差，然后再分别求出 sum 的值就可以了。

比如：

$$\text{sum}[14] = \text{sum}[1110] = C[1110] + \text{sum}[1100]$$

$$\text{Sum}[1100] = \text{sum}[1000] + C[1100];$$

$$\text{Sum}[1000] = C[1000];$$

所以：

$$\text{sum}[14] = C[14] + C[12] + C[8];$$

位运算

虽然我们知道了怎么通过C数组求出sum的值，但是分别找出i对应的C[i]之和比较麻烦。我们都习惯了使用二进制，所以先把十进制转换成二进制，再计算，再转换成十进制特别麻烦。实际上我们可以直接通过二进制来转换

找出二进制数末尾0的个数的办法

$$X \& (-X)$$

X: 二进制数的原码

$\sim X$: 二进制数的反码(原码每位取反)

$-X$: 二进制数的补码(反码+1)

&: 与运算，同1为1，否则为0

|: 或运算，有1为1，否则为0

位运算

比如：7

X : 00000111

$\sim X$: 11111000

$-X$: 11111001

$X \& (-X)$: 00000001

7-1=6

X : 00000110

$-X$: 00000010

$X \& (-X)$: 00000010

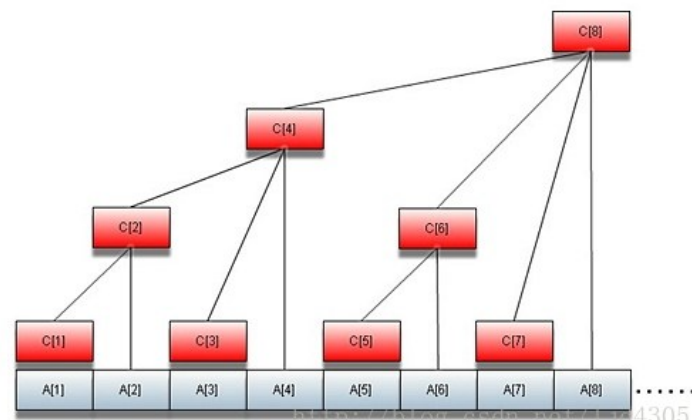
6-2=4

X : 00000100

$-X$: 00000100

$X \& (-X)$: 00000100

4-4=0



$$\text{sum}[7] = C[7] + C[6] + C[4]$$

完全符合

位运算

$X \& (-X)$: 为什么可以求末尾0的个数

设二进制数为 a b c d

那么 $\sim x$ 为 $1-a$ $1-b$ $1-c$ $1-d$

如果 $d=1$

$1-d=0$, $+1=1$

$-x$ 为 $1-a$ $1-b$ $1-c$ d

$\&$ 的结果就为1.

如果 $d=0$, 并且 $c=1$ ($ab10$)

$-x$ 为 $1-a$ $1-b$ c d

$\&$ 的结果就是10.

同理, 如果 $c=0 \dots$

树状数组

第一步：建树状数组 (tree)

第二步：点更新

第三步：求区间之和

实际操作中，第一步和第二步是一样的。建树我们可以理解为已知一个全为0的树状数组，每次让第 i 个点的值增加 $a[i]$ 。

对于第 i 个点，增加 i 的之和，与 i 点一条链上的 $tree[k]$ 的值也会相应的增加 $a[i]$ 。

当 n 个点输入结束的时候，所有的点也就更新好了，时间复杂度为 $O(N * \log N)$ 。

建树状数组

```
void updata(int k, int x) //第k个数增加x
{
```

```
    for(int i=k; i<=n; i+=i&(-i))
    {
```

```
        tree[i]+=x;
```

```
    }
```

比如 $i=1$

$1+x&(-x)=2$

$2+x&(-x)=4$

$4+x&(-x)=8$

这条链上的数刚好为1, 2, 4, 8

```
for(int i=1; i<=n; i++)
{
```

```
    scanf("%d", &a);
```

```
    updata(i, a); //把第i个数增加a(原来为0)
```

```
}
```

区间求和

更新的时候是正着更新，查询的时候反着计算就可以了

```
long long getsum(int x)
{
    long long sum=0;
    for(int i=x;i>0;i-=lowbit(i))
    {
        sum+=tree[i];
    }
    return sum;
}
```

求逆序对

猫猫TOM和小老鼠JERRY最近又较量上了，但是毕竟都是成年人，他们已经不喜欢再玩那种你追我赶的游戏，现在他们喜欢玩统计。最近，TOM老猫查阅到一个人类称之为“逆序对”的东西，这东西是这样定义的：对于给定的一段正整数序列，逆序对就是序列中 $a_i > a_j$ 且 $i < j$ 的有序对。知道这概念后，他们就比赛谁先算出给定的一段正整数序列中逆序对的数目。

本质：求当遍历到第 i 个数 $a[i]$ 的时候，在 i 前面出现了多少个比 $a[i]$ 大的数

求逆序对

当出现一个数的时候，就把这个数标记为1，然后遍历到 $a[i]$ 的时候，求出区间 $[1, i-1]$ 之间有多少个数被标记过，表面上看上去时间复杂度是 $O(N)$ ，但是我们只需要求出区间之和就可以判断在他前面出现了多少个数了。出现过为1，没出现为0. 和即为出现的个数。

区间求和，很明显，用树状数组，每次查询时间复杂度为 $\log(n)$ 。

方法：每出现过一个数 $a[i]$ ，就把 $b[a[i]]$ 标记为1 (++, 可能不止一个)，并且把这条链上的 $tree$ 数组更新 (+1) 然后统计 $sum[a[i]]$ 的个数，表示比他小的，求比他大就是 $i-1-(sum[a[i]]-1)$ 。

离散化

但是，有一个问题，我们是开一个book数组来标记 $a[i]$ ，可能出现这样的情况， $a[i]$ 的值特别分散，比如1 10000 20000000000. 这样空间就会爆，但是实际上只需要占用3个空间，对于n个数来说，最多只需要n个空间。所以我们需要用到离散化思想。

离散化：把无限空间中有限的个体映射到有限的空间中去，以此提高算法的时空效率。比如 $1 \rightarrow 1$, $2 \rightarrow 10000$, $3 \rightarrow 20000000000$ (保持原有大小关系). 这样我就只需要标记1, 2, 3就可以了。

但是对于一个序列来说，比如第 i 个数，我们怎么知道用哪个数字来映射呢？或者说，我们怎知道第 i 个数的大小关系呢？很简单，排序，然后依次用 i 来映射 $a[i].num$.

离散化

一：记录：

```
struct node  
{
```

```
    int num;//第i个数的值
```

```
    int xu;//第i个数的原始位置
```

```
}a[maxn];
```

二：按照num排序

三：记录 $b[a[i].xu]=++k$
注意有重复的数，k的值不变

四：树状数组求区间答案

树状数组用于求解
单点修改，区间极
值比较麻烦，每次
查询时间复杂度为
 $\log n * \log n$ 。一般用
线段树解决

如果需要用到原始值，在开一个映射数组记录

区间修改，点查询

从理论上来说，树状数组只能用于点修改，区间查询。如果用于区间修改，实际上是特别慢的。比如 $[1, n]$ 整个区间的数都+2. 这就需要把所有的 a 的值都+2. 然后重新更新树状数组，效率远低于 $O(n)$ 。

但是在求区间之和的时候，却可以用于区间修改，点查询。但是实质是用单点修改代替区间修改，把单点查询变换为区间之和。

要用到差分思想，区间 $[x, y]$ 都增加 k ，实际上区间内所有的 $a[i+1]-a[i]$ 的值是不会发生任何改变的。改变的只有 $a[x]-a[x+1]$ 和 $a[y+1]-a[y]$ 。所以我们可以把区间修改变为单点修改。把点看成是差分之和。

区间修改区间查询

树状数组的区间修改区间查询稍微复杂一点，其实也比较简单，用习惯了就好，他还是基于差分的思想。

设原数组第 i 位的值为 a_i ， $d_i = a_i - a_{i-1}$ ，则有(这里认为 $a_0 = 0$)：

$$a_x = \sum_{i=1}^x d_i$$

所以有：

$$\sum_{i=1}^x a_i = \sum_{i=1}^x \sum_{j=1}^i d_j = \sum_{i=1}^x (x - i + 1) d_i$$

于是我们得到了：

$$\sum_{i=1}^x a_i = (x + 1) \sum_{i=1}^x d_i - \sum_{i=1}^x d_i \times i$$

区间修改 区间查询

看不懂，没关系，我们来分析一下，设原数组为 $a[i]$ ，差分数组为 $d[i]$ ，前缀和数组为 $c[i]$ 。那么就有

$$a[1]=d[1];$$

$$a[2]=d[1]+d[2];$$

$$a[3]=d[1]+d[2]+d[3];$$

...

$$a[n]=d[1]+d[2]+\cdots+d[n];$$

我们来观察一下 c 数组和 d 数组的关系

对于 $c[i]$ 来说， $d[1]$ 被加了 i 次， $d[2]$ 被加了 $i-1$ 次， $d[i]$ 被加了1次。再根据乘法分配律就得到了：

$$\sum_{i=1}^x a_i = (x+1) \sum_{i=1}^x d_i - \sum_{i=1}^x d_i \times i$$

区间修改 区间查询

所以对于区间修改区间查询的问题，我们只需要维护两个树状数组就可以了，一个维护差分数组 $d[i]$ ，另一个维护 $d[i]*i$ 的值。

```
void updata(int k,int x)
{
    for(int i=k;i<=n;i+=lowbit(i))
    {
        t1[i]+=x;  t2[i]+=(long long)x*k;
    }
}
long long query(int k)
{
    long long ans=0;
    for(int i=k;i>0;i-=lowbit(i))
        ans+=(k+1)*t1[i]-t2[i];
    return ans;
}
```

树状数组

树状数组相对于线段树来说，虽然他们的时间复杂度都是 $n\log n$ ，但是很明显，树状数组的常数要小得多，效率也要快得多，只是我们刚开始使用的时候可能不太习惯，等习惯之后就懒得再去写复杂繁琐的线段树了。

但是树状数组也有他的劣势：

(1) 无法求解区间极值问题，如果当前修改的值就是区间极值，那么修改之后区间的原极值你就知道了。

(2) 树状数组可以用来做简单的区间修改区间查询的题，但是很多比较难的题目中，操作并不是简单的加减，查询的并不是和，这就导致了树状数组很难维护，而线段树就可以很暴力的维护。

二维树状数组

树状数组也有他的优势，就可以能够很快的拓展到二维甚至多维。就拿二维树状数组来说，也就是单点修改，平面查询，可以很快的用二维树状数组解决，如果是二维线段树，不但实现很复杂，而且代码长度翻倍。

而二维树状数组实现很简单，只需要增加一行代码。

二维树状数组

```
void updata(int x, int y, int z)
{
    for(int i=x; i<=n; i+=lowbit(i))
        //在一维的枚举上加一维，三维也是一样
        for(int j=y; j<=n; j+=lowbit(j))
            b[i][j]+=z;
}

long long query(int x, int y)
{
    long long sum=0;
    for(int i=x; i>0; i-=lowbit(i))
        for(int j=y; j>0; j-=lowbit(j))
            sum+=b[i][j];
    return sum;
}
```


必做练习题

洛谷 P3374 树状数组1

洛谷 P3368 树状数组2

洛谷 P3372 线段树1（分别用线段树、树状数组完成）

洛谷 P3373 线段树2

P0J 1195 Mobile phone

二维树状数组其他应用，以及差分实现方法自行学习

线段树

经常用，没什么好讲的，各种模板写熟练就好，除了普通的区间修改，区间查询之外，还需要掌握离散化+权值线段树。

对于线段树的区间修改，除了懒标记下放之外，还有一种常见的做法，标记永久化。他的优势在于效率高，代码简单，并且在树套树中使用很广。

标记永久化

标记永久化的原理很简单，就是用标记的叠加代表这个区间的实际值。比如：

$[1, 5]$ +3

$[1, 2]$ +4

$[2, 4]$ +5

$[2, 5]$ +6

那么如果我们查询点3的值，首先我们访问到 $[1, 5]$ ，此时标记+3，再访问 $[1, 3]$ ，无标记，再访问 $[3, 3]$ ，标记+5+6，标记累加了14，所以点3的值为15。

(自行画一棵 $[1, 5]$ 的线段树模拟)

标记永久化

前面只是区间修改单点查询，那么如果是区间修改，区间查询呢？也可以吗？实际上也是一样，我们除了记录一个`flag`之外再记录一个`sum`，表示当前区间的实际之和。

我们想一下，当区间 $[x, y]$ 和 $+z$ 确定之后，那么包含区间 $[x, y]$ 的线段树的区间的增加的答案也就知道了。即 $z * \text{包含区间长度}$ 。比如 $[1, n]$ ，那么区间之和就增加了 $(y - x + 1) * z$ 。因为标记不下放，所以这里也不下放。

然后查询的时候查询`sum`+当前区间叠加的`flag * 区间长度`就可以了。

标记永久化

```
void update(int id,int l,int r,int x,int y,int val)
{
    tree[id].sum+=(y-x+1)*val;//只更新完整区间
    if(l==x&& r==y)
    {
        tree[id].flag+=val;//标记不下放
        return ;
    }
    int mid=(l+r)/2;
    if(y<=mid)
        update(id*2,l,mid,x,y,val);//l,r 下传, 省空间
    else if(x>mid)
        update(id*2+1,mid+1,r,x,y,val);
    else
    {
        update(id*2,l,mid,x,mid,val);
        update(id*2+1,mid+1,r,mid+1,y,val);
    }
}
```

标记永久化

```
long long query(int id,int l,int r,int x,int y,long long flag)
{
    if(l==x&&r==y)
        return tree[id].sum+(y-x+1)*flag;//flag表从开始叠加到现在的值。
        //sum只是表当前区间以及当前区间包含的区间增加之和的值。
    int mid=(l+r)/2;
    if(y<=mid)
        return query(id*2,l,mid,x,y,flag+tree[id].flag);
    else if(x>mid)
        return query(id*2+1,mid+1,r,x,y,flag+tree[id].flag);
    else
    {
        return ...+...;
    }
}
```

线段树实现

我们知道线段树是非常耗费空间的，同等情况下，花费的空间是其他数据结构的4倍，一般情况下是可以接受的，但是有的时候出题人可能会卡空间，那么我们就需要优化一下空间。

(1) 省略结构体中的 l 和 r ，对于一个点的左右儿子为 $id*2$ 和 $id*2+1$ 这种情况，实际上当 id 确定的时候，区间 $[l, r]$ 也确定了，不需要记录，这就需要我们下传区间。当前 id 涉及区间为 $[l, r]$ ，左儿子区间就为 $[l, mid]$ ，右儿子 $[mid+1, r]$ 。这样可以省掉差不多一半的空间。

(2) 动态开点线段树

在我们学习主席树的时候，我们就用的动态开点线段树，动态开点线段树在高级数据结构中很常见，空间为 $1/4$ 。这就不能省略 l 和 r 了。

线段树实现

```
int build(int l,int r)
{
    int root=++k;//动态开点
    tree[root].sum=0;//目前是空树
    if(l<r)
    {
        int mid=(l+r)/2;
        tree[root].l=build(l,mid);
        //左右子树开好点之后再把该点编号返回给指向他的结点
        tree[root].r=build(mid+1,r);
    }
    return root;
}
```